

Introduction to opps programming assessment

Section A: Concept Application (Theoretical – Scenario Based)

1. Compiler vs Interpreter in Food Delivery Application

In a food delivery application, a compiler is preferred during the final development and deployment stage because compiled programs execute faster and are more efficient for large applications. A compiler checks the whole program before execution, helping detect errors early. This improves performance and reliability when the application is deployed. Interpreters are useful during testing because they execute code line by line, making debugging easier, but they are slower compared to compiled programs.

2. Employee Salary Program

To calculate employee salary with bonus, tax, and overtime conditions efficiently, the following programming constructs should be combined:

- Conditional statements (`if-else`, `switch`)
- Functions for modularity
- Arithmetic operators for calculations
- Loops if multiple employee records are processed

This makes the program organized, reusable, and easy to maintain.

3. Loops in Menu-Driven Application

Loops are preferred because the menu needs to repeat until the user chooses to exit. Using repeated conditional statements would make the code lengthy and inefficient.

A `do-while` loop is most suitable because:

- The menu executes at least once
 - The condition is checked after execution
 - It is ideal for user-driven applications
-

4. Incorrect Average Using int Variables

The probable cause is the use of `int` data type, which stores only whole numbers. Decimal values are truncated during division.

Using `float` or `double` solves the issue because they can store decimal values accurately.

Example:

```
float average = total / 3.0;
```

5. Runtime Error and Debugging

This is a runtime error because the program compiles successfully but crashes during execution.

Two debugging practices:

1. Use print statements or a debugger to trace variable values
 2. Check for invalid memory access, division by zero, or array index errors
-

6. Structure vs Multiple Arrays

Structures are preferred because they keep related data together in one unit.

Example:

```
struct Student {  
    string name;  
    float marks;  
    char grade;  
};
```

Advantages:

- Better organization

- Easier data management
 - Reduces chances of mismatched data
 - Improves readability
-

Section B: Practical Coding Tasks

1. Conditional Logic Implementation

Aim:

To create a Study Mood Assistant using conditional statements.

Program:

```
#include <iostream>
using namespace std;

int main() {
    int hours;

    cout << "Enter hours studied today: ";
    cin >> hours;

    if(hours == 0)
        cout << "Start studying today!";
    else if(hours < 2)
        cout << "Good start!";
    else if(hours < 5)
        cout << "Great progress!";
    else
        cout << "Excellent dedication!";

    return 0;
}
```

Conclusion:

Conditional statements were implemented successfully.

2. Loops + Arrays

Aim:

To calculate total and average screen time using arrays and loops.

Program:

```
#include <iostream>
using namespace std;

int main() {
    float screenTime[7];
    float total = 0, average;

    for(int i = 0; i < 7; i++) {
        cout << "Enter screen time for day " << i + 1 << ": ";
        cin >> screenTime[i];

        total += screenTime[i];
    }

    average = total / 7;

    cout << "Total Screen Time = " << total << endl;
    cout << "Average Screen Time = " << average << endl;

    if(average > 6)
        cout << "Warning: Screen time exceeds healthy limit.";

    return 0;
}
```

Conclusion:

Loops and arrays were used successfully.

3. Functions & Pointers

Aim:

To swap two numbers using pointers and functions.

Program:

```
#include <iostream>
using namespace std;

void swap(int *a, int *b) {
    int temp;

    temp = *a;
```

```

        *a = *b;
        *b = temp;
    }

int main() {
    int x, y;

    cout << "Enter two numbers: ";
    cin >> x >> y;

    cout << "Before Swapping: " << x << " " << y << endl;

    swap(&x, &y);

    cout << "After Swapping: " << x << " " << y;

    return 0;
}

```

Why Pass-by-Reference is Necessary

Pass-by-reference allows the function to modify the original variables directly using pointers.

Conclusion:

Pointers and functions were implemented successfully.

4. File Handling

Aim:

To write and read daily goals using file handling.

Program:

```

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream writeFile("goals.txt");
    string goal;

    cout << "Enter your daily goal: ";
    getline(cin, goal);

    writeFile << goal;
    writeFile.close();
}

```

```

    ifstream readfile("goals.txt");

    cout << "\nSaved Goal:\n";

    while(getline(readfile, goal)) {
        cout << goal;
    }

    readfile.close();

    return 0;
}

```

Conclusion:

File handling operations were performed successfully.

Section C: Mini Capstone Project

Mini Project: Student Productivity Tracker

Aim:

To build a console-based application that tracks student study hours and generates weekly reports.

Program:

```

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    int choice, day;
    float hours, total = 0, average;

    while(true) {

        cout << "\n===== Student Productivity Tracker =====\n";
        cout << "1. Log Daily Study Hours\n";
        cout << "2. Generate Weekly Report\n";
        cout << "3. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch(choice) {

```

```

        case 1: {
            ofstream file("study.txt", ios::app);

            cout << "Enter day number: ";
            cin >> day;

            cout << "Enter study hours: ";
            cin >> hours;

            file << "Day " << day << " : " << hours << " hours\n";

            file.close();

            cout << "Data Saved Successfully!\n";
            break;
        }

        case 2: {
            ifstream file("study.txt");

            string line;

            cout << "\n==== Weekly Report ==== \n";

            while(getline(file, line)) {
                cout << line << endl;
            }

            file.close();
            break;
        }

        case 3:
            return 0;

        default:
            cout << "Invalid Choice!";
    }
}

return 0;
}

```

Conclusion:

The Student Productivity Tracker project was implemented successfully using loops, file handling, and menu-driven programming.